Gridly Group Project - Competitive Crosswords

Team: Name (GitHub Username)

Iago Zamudio Troncoso (iagozamudio)

James O'Connor (jamesoconnr)

Juan Castillero (JuanDCB05)

Samra Redzic (Sare9356)

Thabo Tommy Kelebeng (JoyfulTom)

Xander Chisholm (leBronzo1)

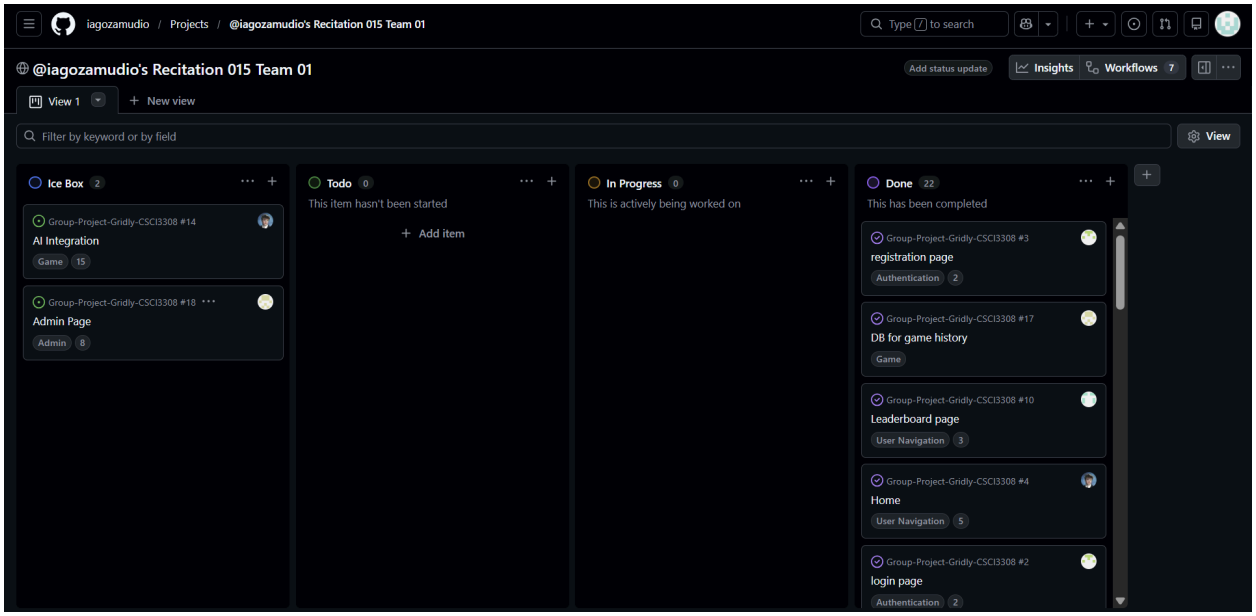
Project Description: Gridly is a competitive, web-based crossword puzzle platform designed for players who want to test their vocabulary and problem-solving skills, either solo or with their friends. Built with Node.js, Express, and PostgreSQL, Gridly procedurally generates unique 5×5 crossword puzzles from a curated clue bank of hundreds of entries, ensuring no two games feel exactly the same.

Players can jump into single-player mode to race against the clock, earning scores based on completion speed and minimal use of hints. For a more competitive experience, the two-player lobby lets users challenge any currently online opponent to a real-time match — both players receive the same puzzle, and a live WebSocket connection tracks opponent progress as the game unfolds. An in-game chat adds a social layer to every match.

Beyond the games themselves, Gridly offers a full user ecosystem: profile pages with game history and personal statistics, an ELO-style rating system that updates after every multiplayer result, and a leaderboard showcasing the fastest times and highest scores across the community. Account management features, including customizable display names, profile pictures, and security-question-based password recovery, round out the experience. Gridly is built for crossword enthusiasts who want more than a puzzle. For those who want rivalry.

Project Tracker:

GitHub project board - https://github.com/users/iagozamudio/projects/1



Demo Video:

https://drive.google.com/file/d/1moD9KeQhPYHpFtY-d5b7x1KFXN7yDDNT/view?resourcekey

VCS:

Gridly GitHub Repository - https://github.com/iagozamudio/Group-Project-Gridly-CSCI3308

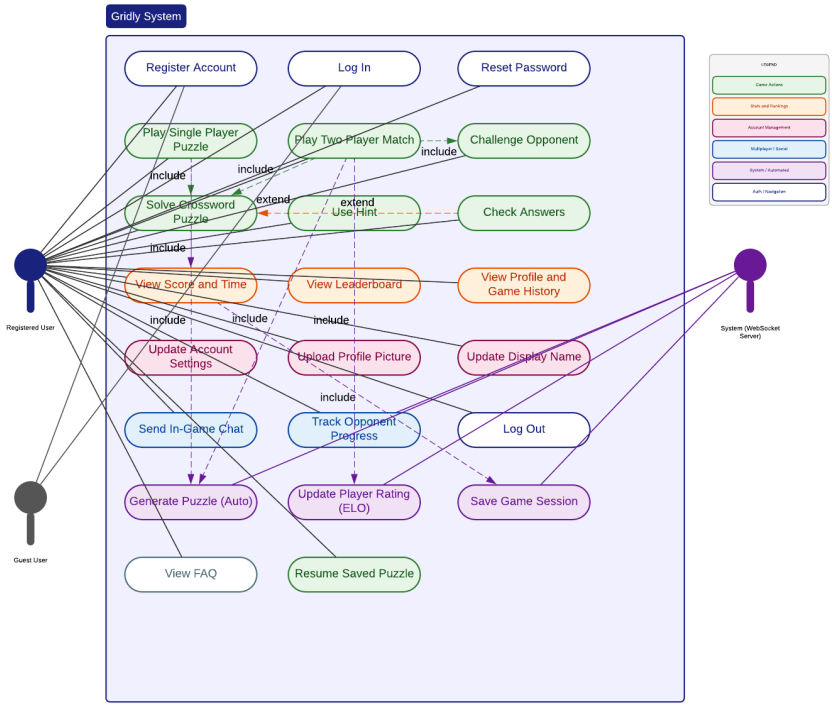
Contributions:

- Iago Zamudio Troncoso: Worked on user authentication and drove password recovery via security questions. Created multiple user tables within the SQL database. Worked on unit testing with Mocha and Chai. Created User Acceptance Test Plan. Worked on the dynamic layout of pages and Handlebar formats. Worked on routing between pages. Deployed PostgreSQL database and Web Service on Render, with the addition of configuring the WebSocket and database for Render deployment. Created the real-time opponent update feature within the two-player mode. Created a double tab within the leaderboard page, which sorted and queried game data.
- James O'Connor: Set up the entirety of the WebSocket server, which is used for all the following listed features. Created chat features for multiplayer. Reorganized flow for crossword creation. Created persistent game sessions. Created a game session table within SQL.
- Juan Castillero: Created the puzzle scoring system for both single player and multiplayer modes and implemented an Elo style player rating system that updates based on game

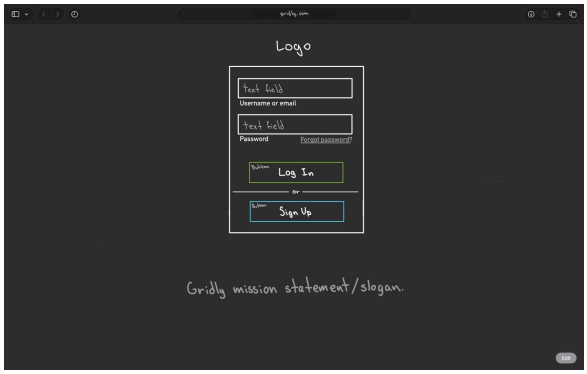
results through WebSocket communication. Built the puzzle gameplay page structure and UI, and created the CSS across most major pages. Created the Gridly logo and initial wireframes. Extended the PostgreSQL schema to support scoring and ratings, integrated these systems with the rest of the application, and merged the local singleplayer and WebSocket based multiplayer versions of the game into a unified codebase while resolving conflicts and bugs.

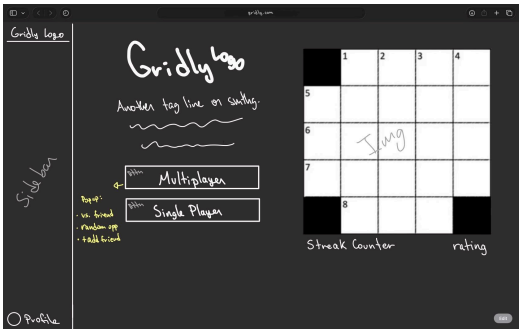
- Samra Redzic: Created profile feature, including game history sub-feature and statistics sub-feature. Created the account settings feature. Accounting settings feature includes: new profile picture feature, change in display name feature, and re-routing to the password reset feature. To ensure a smooth flow of user data, optimized database operations to maintain consistency of user data updates. Ensured dynamic pages by altering the repository structure to use Handlebars to ensure a smoother user experience.
- Thabo Tommy Kelebeng: Created the collapsing chat box feature. Created the majority of the styling for all Gridly pages. Created FAQ page. Created the initial SQL tables for the game logic. Updated Puzzle-answer data for game logic. Created the leaderboard page . Created the initial layout of the homepage.
- Xander Chisholm: Created logic for the game. The same base logic was used to create both the single-player and two-player modes. Created single-player page. Created routing to the single-player pages. Created the original leaderboard page logic where the lowest time was the decider on placement. Created front-end for sign-up and registration pages.

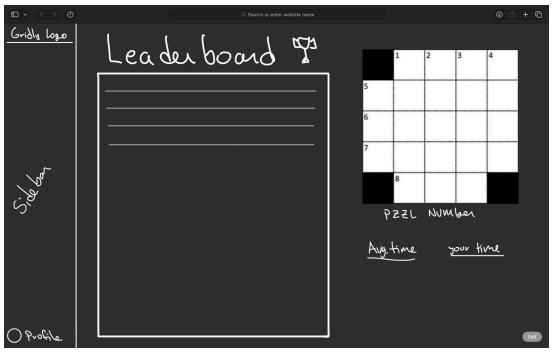
Use Case Diagram:


Gridly System
Register Account
Log In
Reset Password
Play Single Player Puzzle
Play Two Player Match
Challenge Opponent
Solve Crossword Puzzle
Use Hint
Check Answers
View Score and Time
View Leaderboard
View Profile and Game History
Update Account Settings
Upload Profile Picture
Update Display Name
Send In-Game Chat
Track Opponent Progress
Log Out
Generate Puzzle (Auto)
Update Player Rating (ELO)
Save Game Session
View FAQ
Resume Saved Puzzle
include
extend
Registered User
Guest User
System (WebSocket Server)

Wireframes:







Test results:

TC-1.1 Results:

- Pass
- **What did the user do?** The user completed their registration. Afterwards, they were redirected to the login page.
- **What was the user's reasoning for their actions?** They were able to read the prompts for each input field and so complete each correctly.
- **Was the behavior consistent with the use case?** Yes
- **If deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** No

TC-1.2 Results:

- Fail
- **What did the user do?** The user entered an existing username. They were kept on the registration page, but no error message appeared saying "Username already taken."
- **What was the user's reasoning for their actions?** They wanted to see if the system would let them create a duplicate account. The user understood that the registration was unsuccessful but there was no error message.
- **Was behavior consistent with the use case?** Yes
- **If a deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** Yes, we will now implement the error message.

TC-2.1 Results:

- Pass
- **What did the user do?** The user entered their username and password, then clicked Log In. They were redirected to the home page and saw their username displayed.
- **What was the user's reasoning for their actions?** They understood that correct credentials should grant access to the main application.
- **Was behavior consistent with the use case?** Yes
- **If deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** No

TC-2.2 Results:

- Pass
- **What did the user do?** The user entered the correct username but typed the wrong password. They remained on the login page, but they did not see an error message saying "Incorrect username or password."
- **What was the user's reasoning for their actions?** They wanted to test what happens with incorrect credentials. The user understood the login failed because of the error message.
- **Was the behavior consistent with the use case?** Yes

- **If a deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** Yes, we will now make sure the error message shows in this circumstance

TC-2.3 Results:

- Pass
- **What did the user do?** Without logging in, the user typed the home page URL directly into the browser. They were immediately redirected to the login page.
- **What was the user's reasoning for their actions?** They wanted to see if they could bypass the login screen by typing the URL directly.
- **Was the behavior consistent with the use case?** Yes
- **If a deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** No

TC-3.1 Results:

- Pass
- **What did the user do?** The user logged in and navigated to the single-player game page. They observed the 5x5 grid, numbered cells, clue panels, and timer.
- **What was the user's reasoning for their actions?** They wanted to verify that the game interface loads completely before playing.
- **Was the behavior consistent with the use case?** Yes
- **If a deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** No

TC-3.2 Results:

- Pass
- **What did the user do?** The user clicked on a white cell, typed letters, and watched the cursor advance. They also used the backspace key to clear letters.
- **What was the user's reasoning for their actions?** They expected the cursor to move automatically to the next cell, similar to other crossword games they have played.
- **Was behavior consistent with the use case?** Yes

- **If deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** No

TC-3.3 Results:

- Pass
- **What did the user do?** The user filled in one word correctly and one word incorrectly, then clicked the "Check Word" button. Correct cells turned green and incorrect cells turned red.
- **What was the user's reasoning for their actions?** They wanted to confirm that the check tool correctly identifies right and wrong answers.
- **Was the behavior consistent with the use case?** Yes
- **If a deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** No

TC-3.4 Results:

- Pass
- **What did the user do?** The user clicked on an empty cell, then clicked the Hint button. The correct letter appeared, and the cell turned green.
- **What was the user's reasoning for their actions?** They were stuck on a clue and wanted to see if the hint tool would reveal the correct letter.
- **Was the behavior consistent with the use case?** Yes
- **If deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** No

TC-4.1 Results:

- Pass
- **What did the user do?** The user clicked the Logout button while logged in. They were redirected to the login page. When they pressed the browser's back button, they remained on the login page.
- **What was the user's reasoning for their actions?** They wanted to end their session and verify that the back button would not restore access to the home page.

- **Was behavior consistent with the use case?** Yes
- **If a deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** No

TC-4.2 Results:

- Pass
- **What did the user do?** After logging out, the user typed protected URLs (like /home and /single-player) directly into the browser. They were redirected to the login page each time.
- **What was the user's reasoning for their actions?** They wanted to see if they could still access game pages after logging out by typing the URLs directly.
- **Was behavior consistent with the use case?** Yes
- **If a deviation occurred, what was the reason?** No deviation.
- **Did you make changes to the application based on this observation?** No

Deployment: Render Deployment - https://group-project-gridly-csci3308.onrender.com.
Render uses the Project Source Code in addition to a configured PostgreSQL Database. Render automatically sets the environment variables for the database and the WebSocket so that the Web Service can run coherently.
Alternatively, one can clone the GitHub Repository on their local machine. They can then run the project locally through Docker and Docker Compose.

Extra Credit:

Innovative Problem Selection:

i. Target audience: Communities who are passionate about the NYTimes Mini. Communities that want to play with their friends in a competitive setting.
ii. Evidence of problem: The team member's family was interviewed. To protect identity, we will refer to them as X. X stated, "The Mini is cool, but I wanted to race against someone. Also, there was only one."
iii. Existing insufficient solutions: The existing solutions are missing profile features. For those who want to strive for a specific ranking, there should be an available leaderboard. The primary solutions to crosswords don't allow one-on-one racing. Additionally, random generation can be important for competition, which is often missing from crossword puzzles.

iv. Clear differentiation from a basic tutorial-style app: I think it is clear that this is a highly interactive app. It is not a static tutorial app; it is an app where you need to play the game and potentially play against another player.

v. Intentional testing of the application with the target audience and recording of observation: Once the Website was publicly running, the family members were asked to create an account and play the game. They were all able to do so successfully and expressed that they would play if a service like this existed professionally.

Creativity of the Website from the UI/UX perspective

i. Cohesive visual design (e.g.: consistent color themes, palette): The entire website is built in a "dark" theme. Everything from the registration to the leaderboard to the profile contains the same them.

ii. Clear flow and intuitive navigation: The flow is clear because the pages naturally flow into another by the use of the home page and navigation bar. The registration page goes to the login page, which directs you to the home. The home page offers both modes of play, and the navigation bar is persistent throughout all pages.

iii. Custom styling: One example of this is the leaderboard page, which shows multiple leaderboard types by clicking on them. The scores have colors to distinguish them, etc.

iv. Engaging UI elements: We have a custom logo for Gridly, which is displayed in multiple places. The other significant UI elements are all of the dynamic pieces within the game pages. This includes the hint button, check square/line/puzzle buttons, erase button, chat feature, etc.

v. Accessibility considerations: We kept the principles of POUR in mind when creating the page. Although the mobile version of the page still needs further development, we ensured a robust, perceivable, operable, and understandable design.